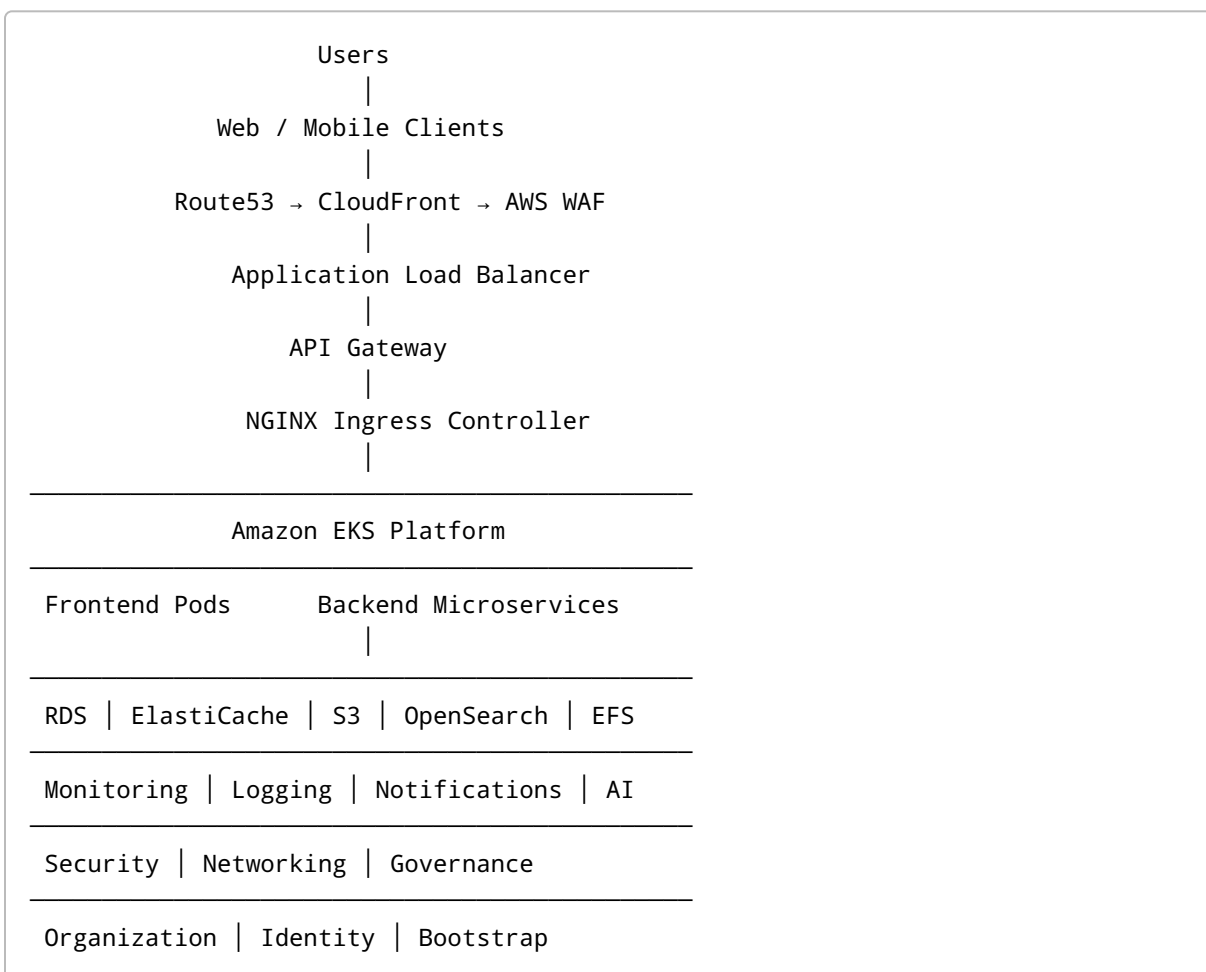


CloudHustler Commerce Platform Architecture (5-Minute Senior Whiteboard Explanation)

When I designed the CloudHustler Commerce Platform, my objective wasn't simply to deploy Kubernetes or AWS resources. The goal was to build an enterprise platform where every layer has a well-defined responsibility and provides services to the layer above it. The platform follows a layered architecture, making it secure, scalable, reusable, and easy to operate.



Phase 1 – Bootstrap (The Foundation)

Every successful cloud platform begins with reliable infrastructure management.

Before provisioning any AWS resources, I created a secure Terraform backend using three reusable modules:

- KMS
- S3
- DynamoDB

The KMS module provisions customer-managed encryption keys used to encrypt Terraform state and other AWS resources.

The S3 module provides a centralized, versioned, encrypted bucket for Terraform state while enforcing bucket ownership, blocking public access, and enabling lifecycle management.

The DynamoDB module provides state locking, ensuring that multiple engineers or CI/CD pipelines cannot modify the infrastructure simultaneously.

Together, these modules create a secure remote backend that every Terraform root module uses throughout the project.

This guarantees consistency, collaboration, and recoverability across the entire infrastructure.

Phase 2 – Identity, Organization, and Governance

Once infrastructure management is established, the next priority is controlling access and governance.

I built a reusable IAM module capable of provisioning:

- IAM Roles
- IAM Policies
- Managed Policy Attachments
- Permission Boundaries
- Instance Profiles
- Standardized Trust Relationships

Rather than embedding raw trust policy JSON everywhere, the module uses structured trust definitions for AWS services, AWS principals, and federated identities. This approach makes IAM reusable, consistent, and easier to maintain.

Using that module, I provisioned dedicated service roles for AWS Config, CloudTrail, GuardDuty, and Security Hub.

Next, I created the AWS Organization layer with Organizational Units for Security, Infrastructure, and Shared Services. This prepares the platform for future multi-account expansion while keeping responsibilities clearly separated.

The Governance layer then deploys CloudTrail and AWS Config.

CloudTrail captures every AWS API call for auditing and forensic investigations.

AWS Config continuously records infrastructure configuration and compliance history.

Together, Identity, Organization, and Governance establish authentication, authorization, auditing, and compliance before any workloads are deployed.

Phase 3 – Networking

With governance in place, I build the networking layer.

The platform uses a highly available VPC spanning multiple Availability Zones.

Each Availability Zone contains:

- Public subnets
- Private application subnets
- Private database subnets

Internet Gateways provide public connectivity.

NAT Gateways allow private workloads to reach AWS services without exposing them directly to the internet.

Security Groups enforce least-privilege communication between every tier.

This network becomes the secure foundation that hosts all application components.

Phase 4 – Edge, Identity, and User Access

Users access the platform through either web or mobile clients.

Amazon Route 53 provides DNS routing.

CloudFront accelerates content delivery globally while caching static assets stored in Amazon S3.

AWS WAF protects the platform from common web attacks, and AWS Shield adds protection against distributed denial-of-service attacks.

User authentication is handled by Amazon Cognito, with IAM and MFA providing secure administrative access.

This creates a secure and highly available entry point into the platform.

Phase 5 – Application Platform

After traffic reaches the platform, requests flow through the Application Load Balancer and API Gateway before entering the Kubernetes cluster through the NGINX Ingress Controller.

The platform runs on Amazon EKS.

Frontend workloads include technologies such as Next.js, React, Redux, and Tailwind CSS running as containerized applications behind Kubernetes Services.

Backend workloads are implemented as independent microservices, including:

- Recommendation Service
- Search Service
- User Service
- Product Service
- Cart Service
- Order Service
- Payment Service
- Inventory Service

Kubernetes provides service discovery, rolling deployments, health checks, self-healing, and horizontal scalability.

This architecture allows each service to evolve independently while maintaining loose coupling.

Phase 6 – Data Platform

Business services rely on multiple storage technologies depending on workload requirements.

PostgreSQL serves as the primary transactional database.

ElastiCache improves application performance by reducing repeated database queries.

Persistent Volumes provide durable Kubernetes storage.

Amazon S3 stores static assets, backups, logs, and data lake content.

OpenSearch provides fast search capabilities and centralized log indexing.

Each storage layer is isolated within private networking and protected using IAM policies, KMS encryption, and Security Groups.

Phase 7 – Observability and Operations

Operating a production platform requires complete visibility.

Prometheus collects application and infrastructure metrics.

Grafana visualizes dashboards for operational monitoring.

Alertmanager routes alerts to engineering teams.

CloudWatch collects AWS service metrics and logs.

Filebeat forwards application logs into the Elastic Stack, where Logstash processes them and Kibana provides centralized log analysis.

This observability stack enables rapid troubleshooting and proactive monitoring.

Phase 8 – Security and DevSecOps

Security is integrated throughout the platform rather than added afterward.

GuardDuty continuously detects threats.

Security Hub aggregates findings across AWS services.

AWS Config evaluates compliance.

CloudTrail records all API activity.

Secrets Manager securely stores application secrets and database credentials.

Within Kubernetes, Kyverno enforces admission policies, RBAC controls authorization, and Trivy scans container images for vulnerabilities before deployment.

This creates a defense-in-depth security model spanning both AWS and Kubernetes.

Phase 9 – CI/CD and GitOps

Application delivery follows a GitOps workflow.

Developers commit code to GitHub.

GitHub Actions build and test applications.

Container images are pushed to Amazon ECR.

Helm packages Kubernetes manifests.

Argo CD continuously monitors Git repositories and synchronizes desired state into the EKS cluster.

This pull-based deployment model eliminates configuration drift and ensures the cluster always matches the declared infrastructure and application configuration stored in Git.

Phase 10 – Event-Driven Processing and AI

The platform extends beyond traditional request-response applications.

Amazon EventBridge coordinates asynchronous events and integrates with third-party vendor systems.

AWS Step Functions orchestrate long-running workflows such as inventory processing, delivery coordination, and customer notifications.

Database changes can flow through AWS DMS into Amazon Kinesis and Firehose before being stored in an S3 data lake for analytics.

Amazon SageMaker and Amazon Bedrock consume this data to build recommendation engines, predictive analytics, and AI-powered business capabilities.

This enables the commerce platform to evolve from transactional processing into real-time intelligence.

How Everything Fits Together

The architecture is intentionally layered.

Bootstrap establishes reliable infrastructure management.

Identity determines who can access resources.

Organization and Governance define operational control and compliance.

Networking provides secure communication.

Edge services securely expose the platform to users.

EKS hosts scalable frontend and backend applications.

The data platform persists business information.

Observability provides operational insight.

Security continuously protects the environment.

GitOps automates application delivery.

Finally, event-driven services and AI extend the platform with advanced business capabilities.

Each layer depends on the services provided by the layer below it while remaining modular enough to evolve independently. This separation of concerns allows the platform to scale across environments, support additional AWS accounts in the future, and continuously deliver new business functionality without redesigning the underlying infrastructure.